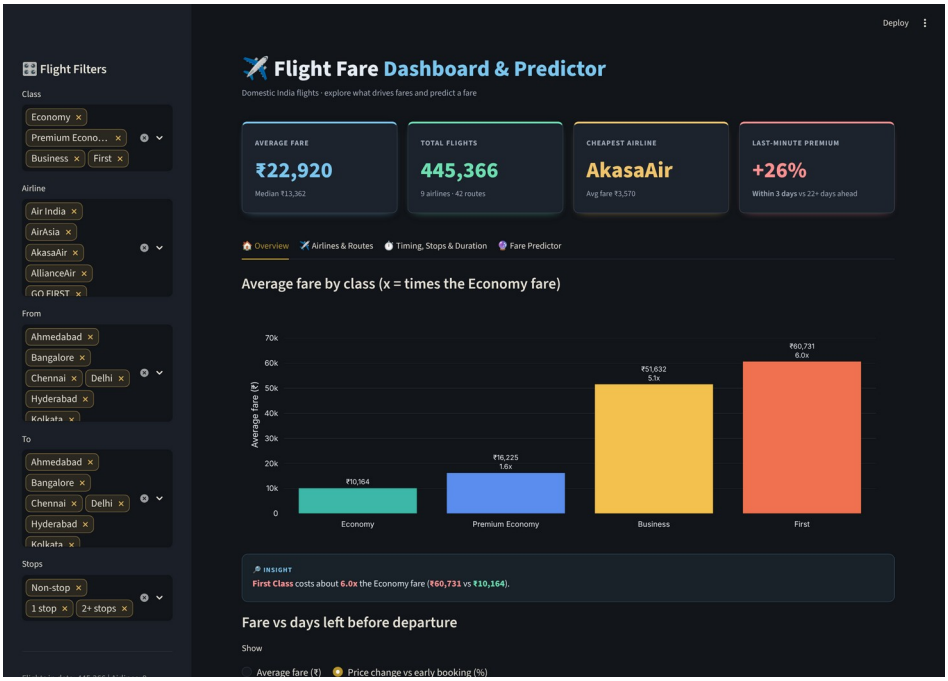
DATA SCIENCE INTERNSHIP PROJECT REPORT

Flight Fare Intelligence

What drives domestic flight fares in India, a model that predicts them, and a deployed dashboard

445,366	9 | 42	R^2 0.909	4 tabs
flights analysed	airlines | routes	on unseen flights	dashboard + predictor



The deployed Flight Fare Dashboard & Predictor

Author	Pratham Rangoonwala
Internship at	Naviotech Solution Pvt. Ltd.
Submitted to	Naviotech Solution Pvt. Ltd.
Date	October 2026
Live app	https://flight-fare-intelligence.streamlit.app/
Code	https://github.com/Pratham719/flight-fare-intelligence

Abstract

Domestic airfares in India vary widely, and travellers have little way to tell whether a fare is high or low for a given trip. This project analyses 445,366 flight fares across seven cities, nine airlines and 42 routes to find out what drives price, and builds a Hist Gradient Boosting model that predicts the fare of a trip (R^2 0.909 and a mean absolute error of ₹3,540 on flights it has never seen).

Class is the strongest driver, followed by flight duration, route and airline. Late booking carries a clear premium: Economy flights leaving within three days cost 49% more than flights leaving 22 or more days ahead. The findings and the model are delivered as a deployed Streamlit dashboard with five filters, KPIs and insight boxes that recalculate with every filter, 14 interactive charts, data-driven booking tips and a fare predictor that responds instantly to every input.

All prices were collected on a single day, so predictions are estimates of what a flight sells for in that snapshot, not live booking prices. The data are fare quotes, not bookings.

Keywords: airfare prediction, India, gradient boosting, Streamlit, interactive dashboard

1. Executive summary

This project analyses 445,366 domestic flight fares across seven Indian cities, builds a model that predicts the fare of a trip, and ships both as an interactive Streamlit dashboard. The questions were simple: what makes a fare high or low, how reliably can it be predicted, and how can a traveller use that.

Key results

- **Class sets the baseline.** Average fares are ₹10,164 in Economy, 1.6x that in Premium Economy, 5.1x in Business and 6.0x in First (only 144 flights).
- **Late booking is expensive.** Economy flights leaving within 3 days cost 49% more than flights leaving 22 or more days ahead (₹13,461 vs ₹9,054). Premium Economy is +89% and Business +22%.
- **Airline matters.** In Economy, Vistara (₹12,082) and Air India (₹11,276) average 38 to 47% more than Indigo (₹8,199); AkasaAir is cheapest at ₹3,570.
- **Prediction is a useful guide, not a quote.** The tuned HistGradientBoosting model reaches R^2 0.909 and a mean absolute error of ₹3,540 on flights it has never seen; 44% of predictions fall within 10% of the actual fare and 70% within 20%. A random split gives higher scores (R^2 0.953, 60.5% within 10%) because the same flights also appear in training.
- **Class dominates the model.** Shuffling Class raises error by ₹14,722, five times more than the next feature (flight duration).
- **The dashboard makes this usable.** KPIs and insight boxes recalculate with every filter, 14 interactive charts and a fare predictor respond instantly, and booking tips are generated from the data (section 6.5).

> **Read with care:** every price in the data was collected on a single day. The fare vs days-left patterns are real within that snapshot, but the model produces estimates, not live booking prices. The data are fare quotes, not bookings, and most flights appear on many dates, so a random split overstates accuracy on new flights.

2. Objective and scope

The project has three goals:

- Explain which factors (class, airline, route, stops, timing, booking window) move fares, and by how much.
- Build and honestly evaluate a model that predicts the fare of a specific trip.
- Deliver the findings as a public dashboard with filters, data-driven booking tips and a fare predictor.

Out of scope: forecasting future prices, real-time booking, and any factor absent from the data such as holidays, fuel prices or seat availability.

3. Data and preparation

3.1 Dataset

The file Cleaned_dataset.csv holds domestic India flight fares for seven cities: Ahmedabad, Bangalore, Chennai, Delhi, Hyderabad, Kolkata and Mumbai. After cleaning it contains 445,366 flights from 9 airlines on 42 routes, departing between 16 January and 6 March 2023, with 1 to 50 days left before departure.

Column	Role	Description
Airline	Feature	Carrier operating the flight (9 airlines)
Class	Feature	Economy, Premium Economy, Business or First
Source, Destination	Features	Origin and destination city; Route = Source → Destination is derived for the dashboard
Departure, Arrival	Features	Time slot: Before 6 AM, 6 AM to 12 PM, 12 PM to 6 PM, After 6 PM
Journey_day	Feature	Day of week of the flight
Stops	Feature	Converted from Total_stops to 0, 1 or 2+
Duration_in_hours	Feature	Total trip duration in hours, including layovers
Days_left	Feature	Days between the price snapshot and departure
Fare	Target	Ticket price in Indian rupees (₹)

Coverage	Flights	Average fare
Economy	248,323	₹10,164
Premium Economy	70,364	₹16,225
Business	126,535	₹51,632
First	144	₹60,731
All classes	445,366	₹22,920 (median ₹13,362)

Table 1. Class mix. The average is pulled up by Business and First; the median is the better single number for a typical fare.

3.2 Cleaning

1. Strip leading and trailing spaces from text columns and convert Fare, Duration_in_hours and Days_left to numbers.
2. Drop rows with missing values and exact duplicates: 6,722 of 452,088 rows (1.5%). In the notebook this happens before the train/test split, so identical rows cannot sit in both sets and inflate the score.
3. Drop Date_of_journey and Flight_code, which are not used.
4. Convert Total_stops to Stops (0, 1, 2+) and add the Route column.

3.3 Important limitation

> All prices were collected on one day. Days_left therefore moves together with the calendar date of departure, and the data cannot show how one specific flight's price changes over time. Findings about booking windows describe the cross-section of flights on that day. The prices are also fare quotes, not bookings, so an average fare is not the average price paid, and the class mix (28% Business, 16% Premium Economy) does not reflect real demand.

4. What drives fares

Unless stated otherwise, comparisons below use Economy only, so that cheap and expensive cabins are not mixed.

4.1 Class

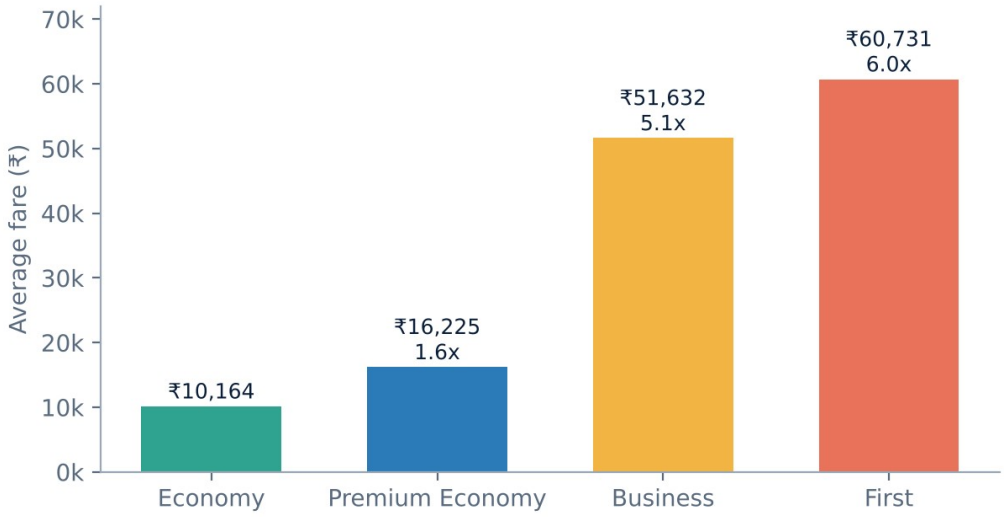


Figure 1. Average fare by class, with the multiple of the Economy fare. First has only 144 flights, all on Air India.

4.2 Booking window

Fares rise as departure approaches, gradually from about three weeks out and most steeply in the final week. Comparing flights leaving within 3 days with flights leaving 22 or more days ahead:

Class	≤ 3 days left	≥ 22 days left	Premium
Economy	₹13,461	₹9,054	+49%

Class	≤ 3 days left	≥ 22 days left	Premium
Premium Economy	₹25,707	₹13,614	+89%
Business	₹60,060	₹49,398	+22%
First	₹61,390	₹61,870	-1% (144 flights)

Table 2. Last-minute premium by class (average fares).



Figure 2. Average Economy fare by days left before departure. The shaded band marks the last three days.

4.3 Airlines and routes

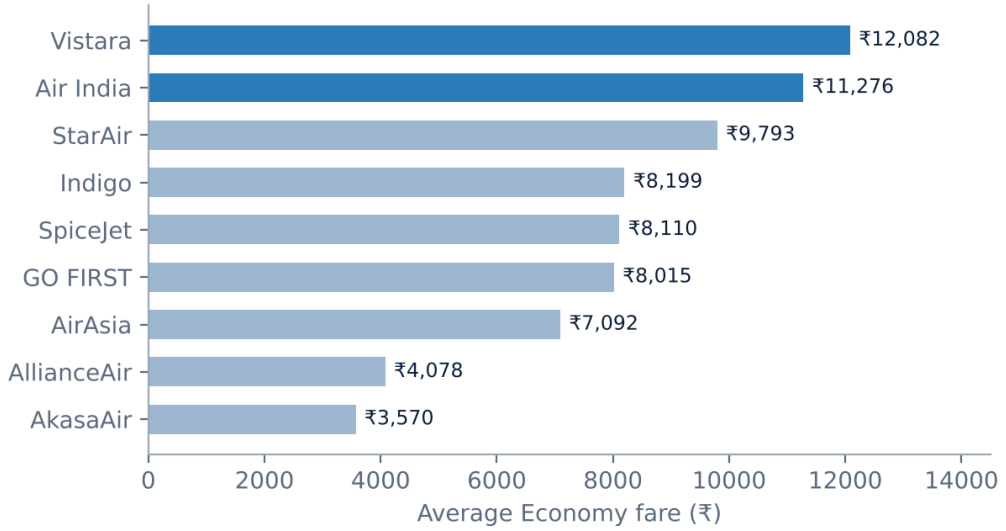


Figure 3. Average Economy fare by airline. StarAir has only 62 Economy flights and AllianceAir 393, so treat their averages as indicative.

- Full-service carriers (Vistara, Air India) are clearly dearer than low-cost carriers (Indigo, AirAsia, GO FIRST, SpiceJet, AkasaAir).
- Fare predictability differs: the middle 50% of AllianceAir fares spans 31% of its median, SpiceJet 42%, Air India 43%, Indigo 57%, AkasaAir 68%.

- Only Air India and Vistara sell Business, and only Air India sells First. Airline averages across all classes therefore overstate their cost: the all-class gap between Vistara (₹27,438) and AkasaAir (₹3,570) is 669%, which is mostly class mix.
- Airline averages also reflect route mix. AkasaAir flies mostly short non-stop routes (average trip 1.6 hours), Indigo averages 5.3 hours, and Vistara and Air India 13.6 and 14.3 hours because they sell mostly one-stop itineraries, so a lower average does not by itself mean better value.
- Costliest Economy routes (30+ flights): Ahmedabad → Kolkata ₹13,338, Ahmedabad → Mumbai ₹13,057, Kolkata → Mumbai ₹12,814. Cheapest: Chennai → Bangalore ₹7,211, Delhi → Bangalore ₹8,549, Bangalore → Delhi ₹8,664.
- Busiest routes: Bangalore → Delhi (10,526 Economy flights), Delhi → Bangalore (10,407) and Delhi → Mumbai (10,276). By origin, Kolkata (₹12,161) and Ahmedabad (₹11,447) are the dearest places to depart from.

4.4 Stops and duration

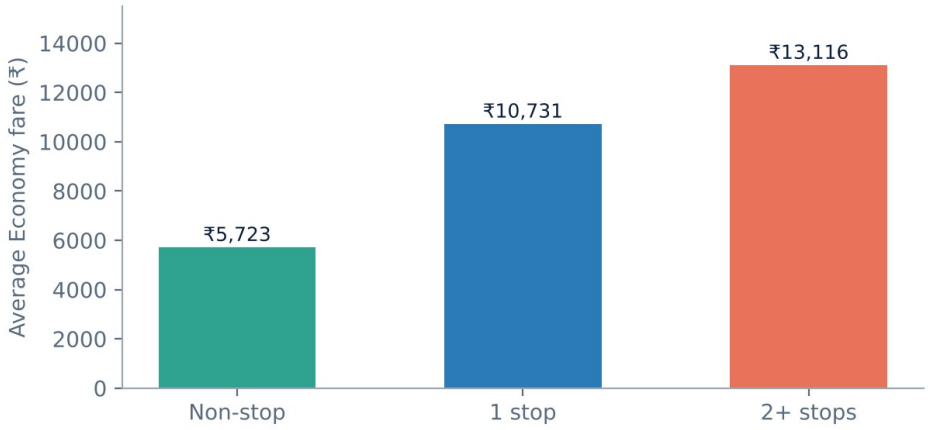


Figure 4. Average Economy fare by number of stops.

Non-stop flights are only 11.6% of all flights (1 stop 82.3%, 2+ stops 6.1%). In Economy, one-stop flights cost 88% more than non-stop (₹10,731 vs ₹5,723) and take about 12.0 hours instead of 2.1. The duration column is total trip time including layovers: on the same routes, one-stop trips take about 11 to 13 hours longer than non-stop (Bangalore to Chennai takes 1.1 hours non-stop and 13.8 with one stop), so duration reflects connection type, not flying time. This is a descriptive gap, not a claim that adding a stop raises the price: the non-stop and connecting flights differ in airline, route and class mix. Across Economy, the correlation between duration and fare is 0.33, a moderate link.

4.5 Timing

- **Departure slot (Economy):** before 6 AM is cheapest (₹9,004), 6 AM to 12 PM is dearest (₹10,696), a gap of 19%. 12 PM to 6 PM averages ₹10,153 and after 6 PM ₹9,693.
- **Arrival slot (Economy):** before 6 AM ₹8,706, 6 AM to 12 PM ₹9,464, 12 PM to 6 PM ₹10,343, after 6 PM ₹10,779.
- **Day of week (Economy):** Tuesday is cheapest (₹9,824) and Sunday dearest (₹10,640), only 8% apart. Because all prices were collected on one day, this also mixes in the days-left effect.

5. Predictive model

5.1 Set-up

The target is Fare. The features are seven categorical columns (Airline, Class, Source, Destination, Departure, Arrival, Journey_day) and three numeric ones (Stops, Duration_in_hours, Days_left). Data were split 80/20, stratified by Class because First has so few rows: 356,292 training and 89,074 test flights, random seed 42. Linear models use one-hot encoding with scaling; tree models use ordinal codes; gradient boosting uses its native categorical support. Fares are right-skewed, so the models learn log(1 + fare) and predictions are converted back to rupees. All metrics below are computed on the rupee scale.

5.2 Model comparison

Model	MAE (₹)	RMSE (₹)	Median AE (₹)	MAPE (%)	R^2
Tuned Hist Gradient Boosting	**2,385.7**	**4,402.3**	**1,069.0**	**10.8**	**0.9531**
Random Forest	2,356.7	4,621.1	873.0	10.8	0.9483
Decision Tree	2,586.9	5,079.5	965.8	12.3	0.9375
Hist Gradient Boosting (default)	3,120.1	5,434.5	1,576.6	14.7	0.9285
Linear Regression	4,710.8	7,543.8	2,589.9	23.7	0.8622
Ridge	4,710.8	7,543.8	2,589.3	23.7	0.8622
Mean baseline	16,724.2	20,323.4	14,223.9	127.3	0.0000

Table 3. Random-split test results for all models.

Tree-based models beat linear regression by a wide margin, so fares depend on interactions between class, airline, route and duration that a linear model cannot capture. Random Forest has a slightly lower MAE (about 1%), but the tuned gradient-boosting model has the best R^2 and RMSE, trains roughly three times faster (about 15 s vs 41 s) and saves to a 5.8 MB file, which suits deployment.

5.3 Tuning

Hyper-parameters were tuned with a 3-fold randomized search on a 100,000-row sample of the training set only; the test set was never touched. The best cross-validated MAE was ₹2,762. The chosen settings were min_samples_leaf 20, max_leaf_nodes 127, max_iter 300, learning_rate 0.2 and l2_regularization 0.1. The final model was then refitted on the full training data and saved as flight_fare_model.joblib with scikit-learn 1.8.0.

5.4 Final performance (random split)

Metric (random split, 89,074 test flights)	Value
R^2	0.953
Mean absolute error (MAE)	₹2,385.7
Root mean squared error (RMSE)	₹4,402.3
Mean absolute percentage error (MAPE)	10.8%
Median absolute percentage error	7.4%
Weighted absolute percentage error (WAPE)	10.4%

Prediction within...	Flights	Share of test set
5% of actual	33,192	37.3%
10% of actual	53,862	60.5%
15% of actual	66,813	75.0%
20% of actual	74,956	84.2%

Table 4. Final metrics and tolerance bands on the random split.

These scores come from a random split. Sections 5.6 to 5.9 repeat the evaluation on whole flights the model has not seen, and those are the numbers to quote.

5.5 Error analysis (random split)

Class	Test flights	MAE (₹)	Mean % error
Economy	49,665	1,213	11.6%
Premium Economy	14,073	1,951	11.0%
Business	25,307	4,927	9.2%
First	29	3,692	5.0% (tiny sample)

Table 5. Test error by class (random split).

Rupee error is larger on expensive cabins but the percentage error is similar (9 to 12%) for the three main classes. In Economy, the highest percentage errors among the large carriers are GO FIRST (14.4%) and Indigo (14.2%). The worst individual misses are cheap fares predicted far too high: the largest is a ₹3,439 fare predicted at ₹17,503, about five times the actual.

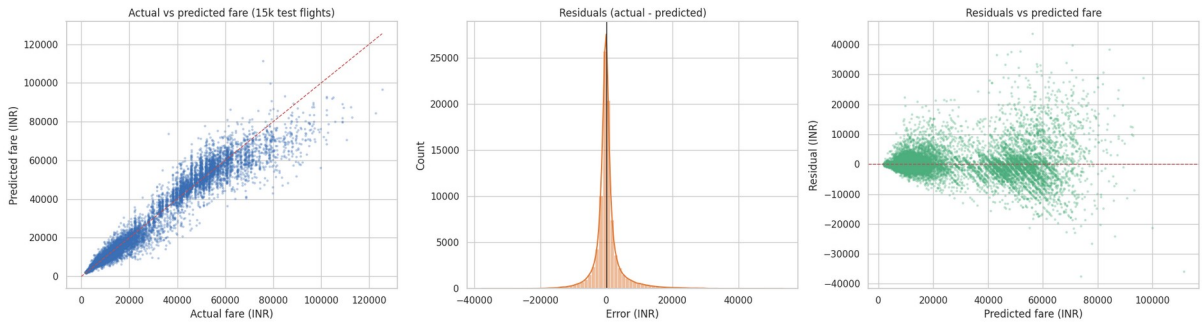


Figure 5. Actual vs predicted fare, residual distribution and residuals vs prediction (from the notebook).

5.6 Validation on flights the model has not seen

The same flight appears on many dates in the data. In a random split, 99.9% of test rows belong to a flight that is also in the training set, so part of the score reflects the model recognising a flight it has already seen. To test genuinely unseen flights, a flight was defined by airline, flight code, route and departure and arrival slots (6,344 flights), and all rows of a flight were kept on the same side of a 5-fold GroupKFold split. The same model settings were used.

Test	R^2	MAE (₹)	MAPE	Within 10%	Within 20%	Baseline R^2
Random split	0.953	2,386	10.8%	60.5%	84.2%	0.922
Unseen flights	0.909	3,540	16.8%	44.4%	69.5%	0.799

Table 6. The same model scored on a random split and on unseen flights. Baseline: the average fare of the same flight and class in training (the class average when the flight is new).

On unseen flights R^2 falls from 0.953 to 0.909, and the share of predictions within 10% of the actual fare from 60.5% to 44.4%. The model still beats the baseline (0.909 vs 0.799), so it is learning price patterns and not only recognising flights. The unseen-flight numbers are the ones to quote. A split by departure date was also run, but because all prices come from one day, 99.9% of its test flights also appear in training, so it is not a true time test and is not reported.

5.7 Uncertainty by class and airline

A single score hides how uncertain it is, especially for small groups. 95% confidence intervals were computed by resampling whole flights 1,000 times (rows of one flight are not independent), using the unseen-flight predictions.

Class	Rows	Flights	MAE (₹)	MAE 95% CI	Average miss	Within 10%
Economy	248,323	6,090	1,814	1,776 to 1,850	18.1%	41.8%
Premium Economy	70,364	1,488	2,822	2,728 to 2,918	17.0%	45.1%
Business	126,535	2,510	7,306	7,098 to 7,534	14.2%	49.4%
First	144	4	20,866	13,860 to 42,072	31.8%	0.0%
All classes	445,366	6,344	3,540	3,460 to 3,622	16.8%	44.4%

Table 7. Error on unseen flights by class. Average miss is the mean absolute percentage error.

- 95% intervals for the average miss: Economy 17.7 to 18.5%, Premium Economy 16.3 to 17.7%, Business 13.8 to 14.7%, First 27.0 to 44.3%.
- Business is the most reliable class in percentage terms and Economy the least. The rupee error is largest in Business because its fares are higher.
- First class has 144 rows from only 4 flights. Its interval is very wide and none of its predictions fall within 10%, so its estimates should not be relied on.

Airline	Rows	Flights	Average miss (95% CI)	Within 10%
Air India	118,405	1,605	14.5% (13.9 to 15.2)	53.1%
AkasaAir	2,230	54	15.7% (13.5 to 18.3)	46.8%
Vistara	226,204	1,901	17.2% (16.7 to 17.9)	42.3%
AirAsia	15,764	353	17.8% (16.8 to 19.0)	38.5%
SpiceJet	5,863	227	18.2% (16.2 to 20.3)	40.7%
Indigo	65,659	1,869	18.6% (18.0 to 19.2)	39.5%
GO FIRST	10,786	319	21.5% (19.8 to 23.2)	35.0%
AllianceAir	393	12	15.2% (12.0 to 19.1)	39.7%
StarAir	62	4	29.6% (23.1 to 40.4)	11.3%

Table 8. Error on unseen flights by airline. AllianceAir and StarAir have too few flights to judge.

Airlines are compared by percentage error because rupee errors scale with fare level and route mix. Air India is the most accurate (14.5% average miss, 53.1% within 10%) and GO FIRST the least (21.5%, 35.0%).

5.8 Systematic error

Average error can look small while one group is consistently over or under priced. The average signed error on unseen flights (positive means the model prices too high) was checked by class, airline, route and days-left bucket.

- No meaningful bias for most groups: average errors by class, airline and days-left bucket stay between about +1% and +3%, slightly too high.
- First class is the exception: the model prices it about 32% too low on average (-31.8%).
- Among routes with at least 300 rows, the largest over-pricing is Chennai to Bangalore (+8.6%). No route is under-priced by more than 1%.
- Errors are slightly larger close to departure: +3.1% within 7 days against +1.6% at 31 or more days.

5.9 Prediction intervals

Instead of a single number, a prediction interval gives a range that should contain the real fare most of the time. Split conformal prediction was used: the model was trained on one set of flights, its typical miss on the log fare scale was measured on a separate calibration set of flights it had not seen, and that miss was used to put a range around each new prediction. First class is pooled with Business because it has too few flights. Training used 257,561 rows, calibration 96,862 and testing 90,943, with no flight in more than one set.

Class	Test rows	Test flights	Fares inside range	Range width
Economy	50,898	1,220	76.3%	57%
Premium Economy	14,316	292	73.3%	52%
Business	25,652	510	76.7%	45%
First	77	2	64.9%	45%
All classes	90,943	1,269	75.9%	57%

Table 9. Coverage of an 80% prediction range on unseen flights. Range width is the median of (high - low) as a share of the predicted fare. First class has 2 test flights, so its row is not reliable.

The target was 80% of real fares inside the range and the result was 75.9%, so the range is best described as "about 3 in 4". The ranges are wide: about 57% of the predicted fare in Economy and 45% in Business. For a predicted Economy fare of ₹10,000 the range is roughly ₹7,500 to ₹13,200. This matches the finding that only 44% of predictions fall within 10% of the actual fare. The ranges were tested in the evaluation notebook and are not yet shown in the dashboard.

5.10 Feature importance

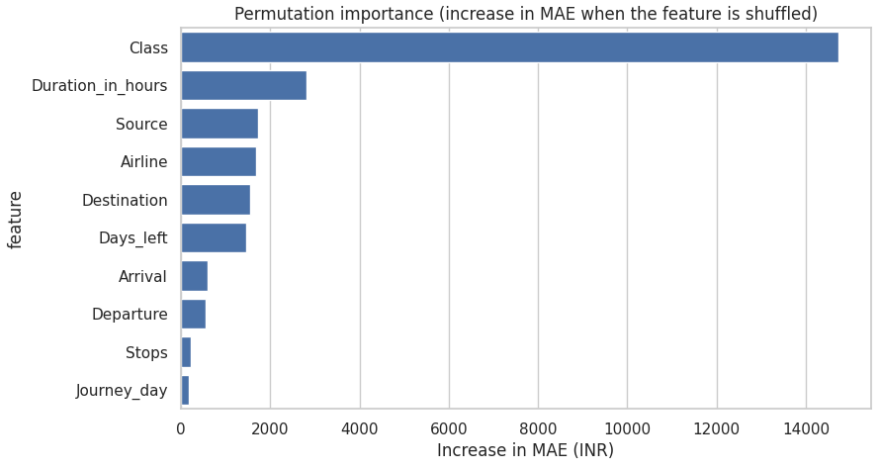


Figure 6. Permutation importance: increase in MAE (₹) when a feature is shuffled, on 20,000 test flights.

- **Class** (₹14,722) dominates, about five times the next feature.
- **Duration** (₹2,815) is second. It is total trip time including layovers, so it partly captures connection type. **Source** (₹1,719) and **Destination** (₹1,555) together make the route a major driver, close behind duration and ahead of **Airline** (₹1,671 on its own).
- **Days left** (₹1,459) matters about as much as airline.
- Departure slot, arrival slot, stops and weekday are each under ₹600. Stops adds little once duration is known, because connecting flights are the long flights.

5.11 How to read the predictions

- The model learned from one day's prices, so it estimates what a flight with these attributes sells for in that snapshot, not what it will cost on a future date.
- A random split overstates accuracy because similar flights appear across dates. Use the unseen-flight results in sections 5.6 and 5.7 (R^2 0.909, 44.4% within 10%). The dashboard shows the typical miss for the chosen class under every prediction.
- The predicted-fare-vs-days-left curve in the dashboard is jagged rather than smooth because days left is entangled with calendar date in the training data.
- If the saved model fails to load, the app trains a fallback model of the same type on a 200,000-row sample, so its accuracy can differ slightly. The sidebar always shows which model is in use.

6. The dashboard

6.1 Overview

The app is built with Streamlit and Plotly, uses a dark theme and adapts to tablet and phone widths. A sidebar offers five filters (Class, Airline, From, To, Stops). Above the tabs, a row of hero KPIs shows the average fare, total flights, cheapest airline and the last-minute premium for the current selection. Four tabs follow, containing 12 analysis charts and 2 predictor charts, one chart per row; radio buttons appear only where charts share axes. The sidebar ends with an About this data note (scope, limits and the model in use) and links to the author's LinkedIn, GitHub and a fork page.

6.2 Tab by tab

Overview. Average fare by class (as a multiple of Economy), fare vs days left (switch between average fare and % change vs early booking), and the fare distribution (histogram or box plot by class).

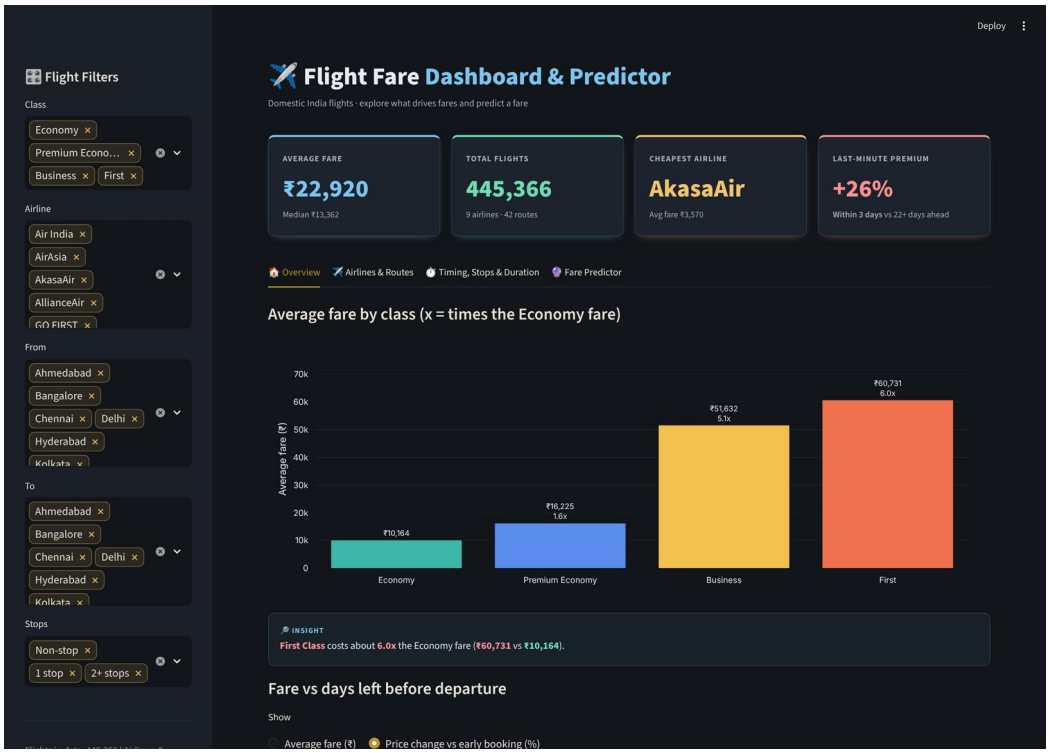


Figure 7. Overview tab with hero KPIs.

Airlines & Routes. Four KPI cards (costliest airline, airline price gap, costliest and cheapest route; the airline cards also show the average trip time, because airline averages reflect route mix) and five charts: average fare by airline, fare spread by airline, route heatmap, top 10 routes (costliest, cheapest or busiest) and class mix by airline.

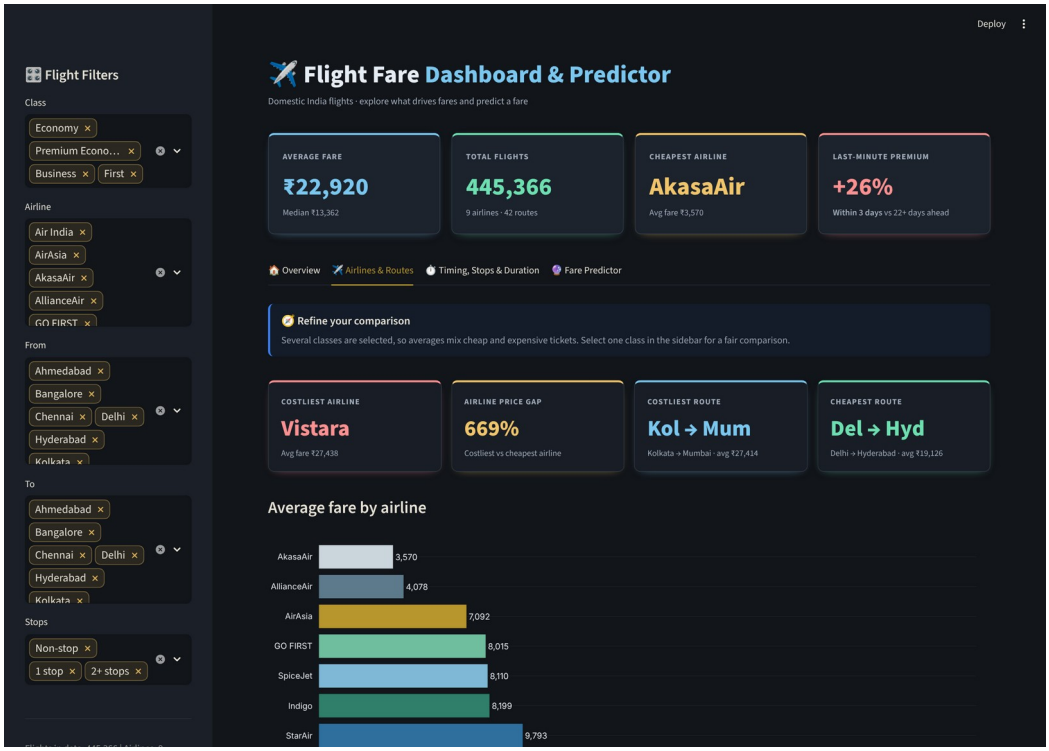


Figure 8. Airlines & Routes tab. With all classes selected, a "Refine your comparison" note explains why airline averages mix cheap and expensive tickets.

Timing, Stops & Duration. Four KPI cards (non-stop share, average duration, connecting-flight premium, cheapest departure slot) and four charts: fare by departure or arrival time, by day of week, by number of stops, and duration vs fare.

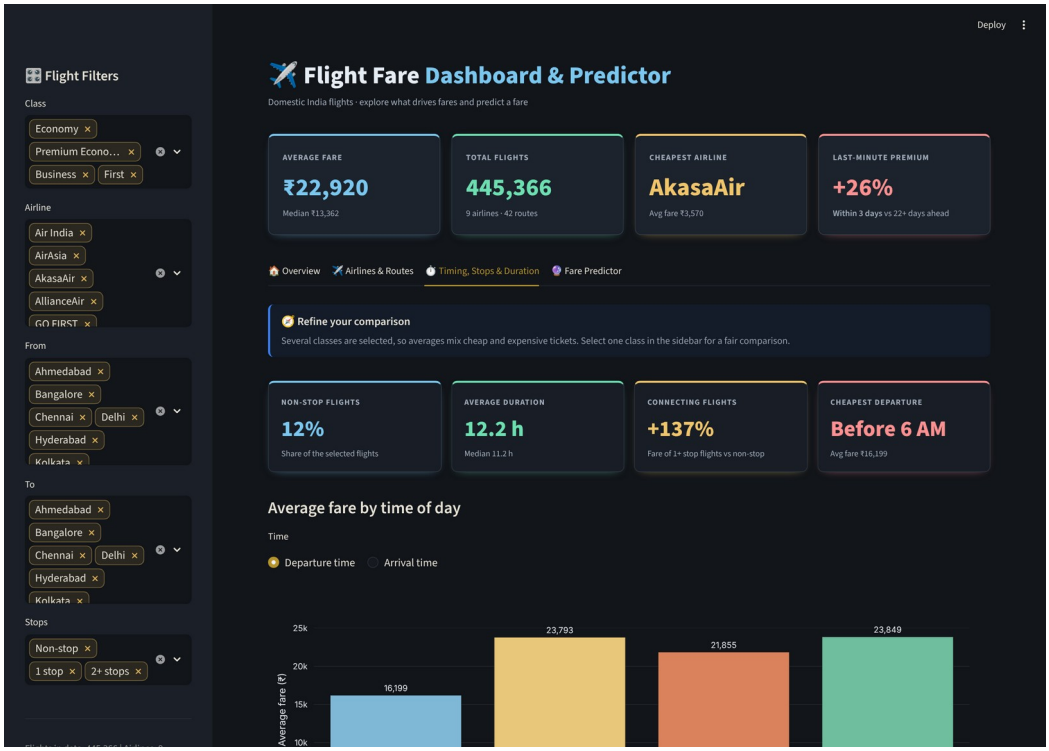


Figure 9. Timing, Stops & Duration tab.

Fare Predictor. Inputs for class, route, airline, stops, days left, departure and arrival time, day and duration. It returns the predicted fare, the route median, the middle 50% of fares on that route (not

a prediction interval) and the difference from the median, with the typical miss for the chosen class shown underneath (First class carries a warning), plus a predicted fare vs days-left curve and the same trip on every airline selling that class.

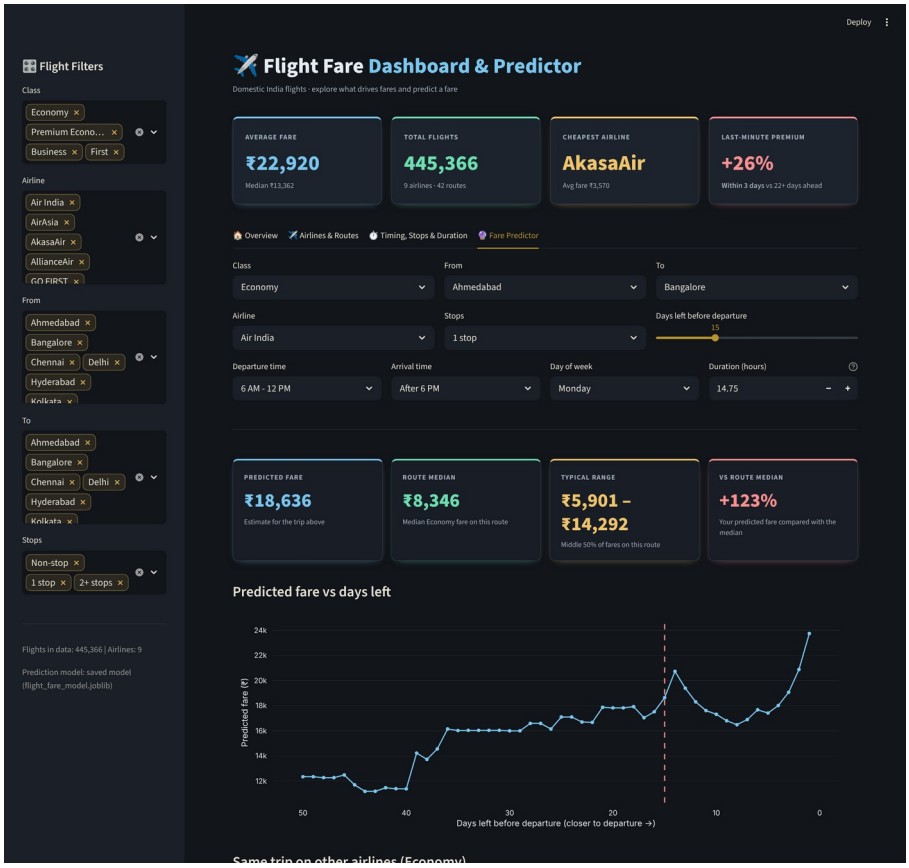


Figure 10. Fare Predictor tab (Economy, Ahmedabad → Bangalore, Air India, 1 stop, 15 days left).

6.3 Filters in action

Changing a sidebar filter recalculates the KPIs and the insight text. Selecting Economy only moves the average fare from ₹22,920 to ₹10,164, flights from 445,366 to 248,323 and the last-minute premium from +26% to +49%.



Figure 11a. All classes selected.



Figure 11b. Class filter set to Economy only.

6.4 What is dynamic and what is not

Component	Follows sidebar filters?	Behaviour
Hero KPIs and tab KPI cards	Yes	Recalculated from the filtered data on every change. The last-minute premium compares fares within 3 days to fares 22 or more days ahead.
Charts	Yes	All charts use the filtered data, except Class mix by airline, which deliberately ignores the Class filter.
Insight boxes (🔎)	Yes	Numbers and names are generated from the filtered data. A few explanatory phrases are fixed text, and the model-accuracy note on the predictor tab is static.
Smart Booking Tips	No	Generated automatically from the full dataset (airline names, routes and percentages come from the data, nothing is hardcoded) and intentionally independent of the sidebar. Up to four cards on Overview, Airlines and Timing. On Airlines and Timing they change with a class selector, Economy by default.
Fare Predictor	Own inputs	Every output updates instantly on any input change. Duration auto-fills with the typical value for the chosen airline, class, route and stops.
Warnings	Automatic	“Refine your comparison” appears on Airlines and Timing when several classes are selected; a low-confidence note appears when the chosen airline, class, route and stops combination is not in the data; a warning appears if the filters leave no flights.

Table 10. Dashboard behaviour.

6.5 Key advantages

The dashboard is designed so that a traveller or analyst gets an answer without writing a query or reading raw tables. Its main advantages are:

Feature	What it does	Benefit
Dynamic KPIs	Hero KPIs (average fare, total flights, cheapest airline, last-minute premium) and the tab KPI cards recalculate from the sidebar filters.	A headline answer for any segment, such as Economy on one route, in one click.
Dynamic insight boxes (🔎)	Under each chart, a short sentence is generated from the filtered data, with changing numbers, airline names and class names.	The takeaway is stated in plain words, so charts do not need to be decoded.
Interactive charts	14 Plotly charts with hover details and toggles: average vs % change, histogram vs box plot, costliest / cheapest / busiest routes, departure vs arrival time.	Explore the same data from several angles without leaving the page.

Feature	What it does	Benefit
Five sidebar filters	Class, Airline, From, To and Stops apply to every tab at once.	One consistent view across all charts and KPIs.
Smart Booking Tips	Up to four tips per tab, written automatically from the full dataset with a class selector on the Airlines and Timing tabs. Nothing is hardcoded.	Actionable advice that stays correct if the data is refreshed.
Fare Predictor	Updates instantly on any input, auto-fills duration, shows the route median, the middle 50% of route fares, the typical error for the class, a fare vs days-left curve and the same trip on every airline.	Checks whether a fare is high or low and which airline is cheapest for the same trip.
Smart warnings	Notes appear automatically when several classes are mixed, when a combination is not in the data, or when filters return no flights.	Prevents misleading comparisons and over-trusting weak estimates.
Mobile-friendly design	Dark theme, responsive KPI and tip grids, short city codes on charts and legends that wrap on small screens.	Readable on a phone as well as a laptop.
Fast on 450k rows	Cached data and model, pre-binned histogram, sampled box and scatter plots, and per-tab fragments.	Interactions stay quick on the free hosting tier.
Transparent	An About this data note in the sidebar explains the data scope, its limits and which model is serving, and the predictor states that prices are one-day estimates.	Users can see what they are relying on.

Table 11. Key dashboard advantages.

7. Engineering challenges and solutions

Problem	Cause	Solution
Memory	450k rows copied on every click ran the app out of memory.	One cached copy of the data and model (st.cache_resource); filtered frames keep only the 12 columns needed; the histogram is pre-binned so the browser receives 50 rows; box and scatter plots use samples (up to 10,000 points); each tab is a Streamlit fragment.
Model loading	The saved model failed with a scikit-learn version mismatch (_RemainderColsList error).	Re-saved the model in Colab with the same version as the app (1.8.0).
Deployment	On Streamlit Cloud, a second error appeared: No module named '_loss'.	Pinned exact package versions in requirements.txt.
Debug visibility	Hard to tell which model was serving after a silent failure.	The sidebar shows whether the saved or the fallback model is in use; the fallback is a matching HistGradientBoosting pipeline trained automatically.
Repo hygiene	Unused .pkl model files cluttered the repository.	Removed them; only the joblib model remains.

8. Deployment and reproducibility

The app is deployed on Streamlit Cloud from GitHub at https://flight-fare-intelligence.streamlit.app/ (code at https://github.com/Pratham719/flight-fare-intelligence). The repository contains app.py,

Cleaned_dataset.csv, the notebooks 01_flight_fare_prediction.ipynb (EDA and modelling) and 02_evaluation.ipynb (unseen-flight evaluation, confidence intervals, bias checks and prediction intervals) in notebooks/, the model in model/flight_fare_model.joblib with a model card in model/README.md, a .streamlit/config.toml settings file, requirements.txt, a README and a licence.

Package	Version
joblib	1.5.3
numpy	2.4.1
pandas	2.3.3
plotly	6.7.0
scikit-learn	1.8.0
scipy	1.17.1
streamlit	1.55.0

Table 12. Pinned requirements.

To run locally: install the requirements with pip, then run streamlit run app.py from the repository folder. To rebuild the model, run notebooks/01_flight_fare_prediction.ipynb top to bottom in Google Colab with scikit-learn 1.8.0 and re-save flight_fare_model.joblib. To reproduce the evaluation in section 5.6 to 5.9, run notebooks/02_evaluation.ipynb next to Cleaned_dataset.csv.

9. Limitations

- All prices come from a single day, so the data cannot separate days-left from calendar date, and the model cannot learn how prices move over time.
- Departures span only about seven weeks and exclude major festivals and holiday periods.
- First class has 144 flights, all on Air India; its results are not reliable. On unseen flights its error is about 32% and the model prices it about a third too low.
- A random split overstates accuracy because nearly every test flight also appears in training. The unseen-flight scores in section 5.6 are lower, and a split by date is not possible with one day of prices.
- Some airlines have very few Economy flights (StarAir 62, AllianceAir 393), so their statistics are indicative only.
- The prices are fare quotes, not bookings, so average fares are not what travellers paid, and the class mix (28% Business, 16% Premium Economy) is not real demand.
- Duration includes layover time (one-stop trips take about 11 to 13 hours longer than non-stop on the same routes), so it reflects connection type more than flying time.
- Airline averages reflect route mix and class mix, so a lower average does not mean a cheaper airline for the same trip.
- Prediction ranges covered about 76% of real fares against an 80% target, and are not yet shown in the dashboard.

10. Conclusions and next steps

Fares in this dataset are driven first by class, then by flight duration, route and airline, with booking window a close next. Late booking carries a clear premium in Economy and Premium Economy, while timing slots and weekday matter far less. A tuned gradient-boosting model predicts fares with R^2 0.909 on flights it has not seen (average error about ₹3,500, with 44% of predictions within 10% of the actual fare), and the dashboard turns these findings into something a traveller can filter, compare and act on, with live KPIs, insight boxes, booking tips and warnings that keep the comparison fair.

Next steps

- Collect fares on many different days so the model can learn real price movement over time. This is the biggest lever: more rows from the same day will not help much.
- Add public holidays, fuel prices and seat availability.
- Show the prediction ranges (section 5.9) in the dashboard, and report First class pooled with Business or with a very wide range.
- Add a time-based split once fares from several days exist, and compare XGBoost and LightGBM.
- Track runs with MLflow, version the data and model with DVC, and add a CI smoke test for the app.